

Certificate Management

Overview

Rhapsody supports receiving and sending messages in a networked environment utilizing a range of protocols:

- TCP
- FTP / FTPS
- SFTP
- HTTP / HTTPS
- Web Services

Communications over network connections are clear text by default (that is, they are not encrypted and therefore rely on the security of the general network to maintain data privacy). Rhapsody provides the ability to transfer messages over secure network connections utilizing either SSL or SSH encryption systems, as supported by the communication point and underlying protocol. Rhapsody also provides a number of filters that are able to encrypt, decrypt, or sign messages.

This lesson covers the management of the certificates needed to support these encryption and signing operations.

Certificate Management

Configuring a secure connection in Rhapsody requires the following steps:

1. Generate (or import) the required keys and certificate in the Rhapsody Certificate Manager.
2. 2

Securely manage distribution of certificates and keys containing the encryption key to parties with whom communications will be established.

3. 3

Securely obtain the relevant certificate from the party with whom you wish to establish communications and import to the Certificate Manager.

4. 4

Configure the communication point/s and filters to use the certificate and keys.

Certificates and Keys

There are two general approaches to cryptographically secure information: Symmetric key and Public key. A brief introduction of the two approaches is covered below.

Symmetric Key Cryptography

With Symmetric Key cryptography, a message is combined with a key in an algorithm to generate an encrypted message. The original plain text message is recovered by using the same key with the algorithm to perform the decryption. A symmetric key algorithm is designed to make it hard for someone to determine the plain text version of a message without access to the correct key.

The disadvantage of symmetric key cryptography relates to key distribution; both the sender and the recipient must have the same key. Secure communications can be intercepted and decrypted if an unauthorized third party obtains a copy of the key.

Public Key Cryptography

With Public Key cryptography, a **public key** (or certificate) is used in conjunction with an algorithm to encrypt a message. A separate **private key** is required to decrypt the message. This makes it possible to openly distribute the public keys in the sure knowledge that they cannot be used to decrypt a message. If Bob, for example, wishes to send a secure message to Alice, he will encrypt it using Alice's public key. Alice will in turn decrypt the message using her private key.

Public key algorithms can be run backwards. For example, a message encrypted with the sender's private key can be decrypted with the sender's public key. This is used to digitally sign messages, which allows someone with access to the sender's public key to confirm the identity of the sender, as only he or she has their private key.

The disadvantage of public key cryptography is that the key must be significantly more complex than a symmetric key for the same level of security. This in turn has an impact on performance. Some protocols (such as SSL) will use a public key during the establishment of a connection to securely share a private key that is used for the remainder of the communication.

Certificate Authorities

Validation of the credentials and keys is managed by the issuing of Certificates by accredited organizations which are recognized as trustworthy and able to validate identity. These organizations are called Certificate Authorities (CA) and issue certificates which:

- Are signed by the CA with a key identifiable as their key (that is, the certificate can be validated as issued by the CA).
- Contain the public key of the holder.
- Contain the identifying details of the holder.
- Verify that the public key is that of the holder.

In cases where the two parties are known to each other and agree to recognize each other, Self Signed certificates may be used. In these cases, each party generates their own certificate and shares the certificate with the other party. The content of the certificate is the same, except that the Authority Identifier is the issuing organization.

The Rhapsody Certificate Manager provides the ability to place Certificates and Private Keys into a managed store, and makes the keys available to the components which require them.

Encryption of a connection requires two keys:

- Private key: this remains private to the recipient and is never disclosed. This key is used to decrypt a message encrypted with its matching public key.
- Public key: provided by potential recipients to all senders. This key is utilized to encrypt a message that can only be decrypted by the private key. The distribution of a public key is controlled by its owner.

Public keys are normally controlled by distributing the Certificate containing the public key. The private key must never be shared. This Certificate establishes a chain of trust to authenticate that the key identifies communications from the certificate holder and includes the public key, the identity of the issuing authority and the identity of the certificate holder.

Security Considerations

When considering how a secure interface will be constructed in Rhapsody, you will need to consider a number of factors. These include:

- The cipher used will depend on the supported ciphers in the systems you are connecting to and current best practices. These change over time as weaknesses in old ciphers are identified and new ciphers are developed. When working with some communication points (for example, the HTTP Client or (S)FTP Client communication point), you can specify a desired level of security in your configuration, which defines a list of ciphers that will be supported when connecting to an external system. For the connection to succeed, the external system must support one of the ciphers in the list. For keys generated using the

Rhapsody Certificate and Key manager, you need to check with the external system to understand what ciphers are supported when creating the key.

- Different ciphers require different lengths of key for the same effective level of security. Generally, a public key cipher requires a much longer key than a symmetric key cipher.
- The presence of any hardware or operating system level cryptographic features (such as a VPN link). Where the connection being used by Rhapsody can be considered cryptographically secure, you may be able to reduce the degree of communication cryptography implemented in Rhapsody.
- Avoid the temptation to roll your own cryptographic solution. Getting cryptography correct is very difficult. Almost all custom solutions contain problems that prevent them from being secure. Always use well known and widely used industry standards to secure your communications.

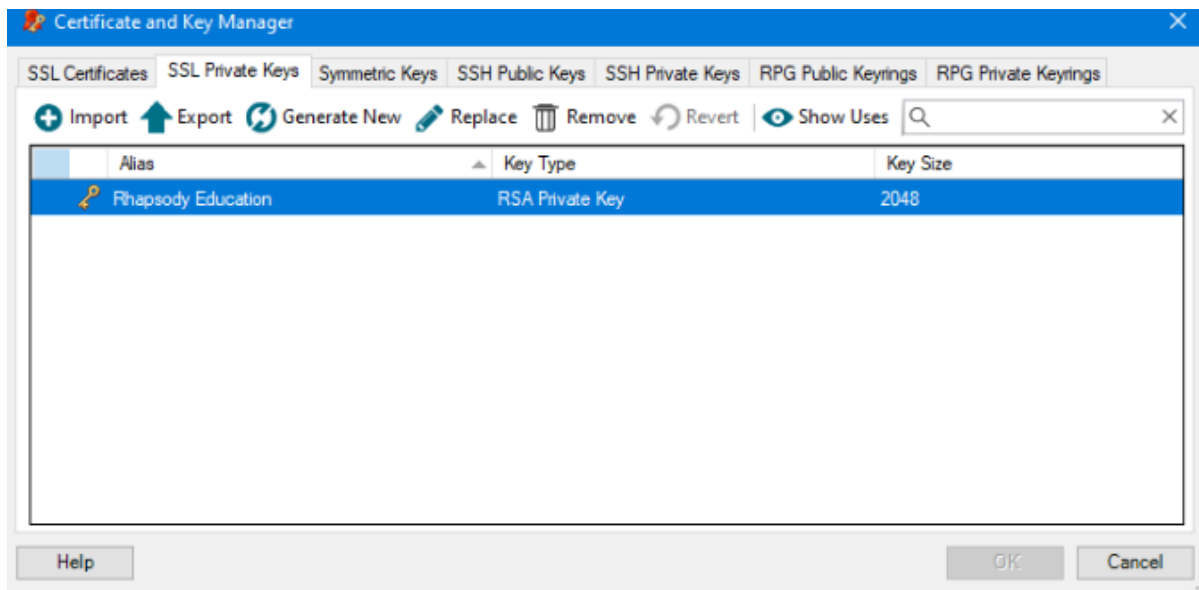
Further Reading

[\(opens in a new tab\)](#)

- [Symmetric-key algorithm\(opens in a new tab\)](#)
- [Public-key cryptography\(opens in a new tab\)](#)
- [Strong cryptography](#)

The Certificate and Key Manager

The Rhapsody IDE incorporates the Certificate and Key Manager, accessible from the View menu item or from the Managers drop-down. It is used to import, export, create, update and delete certificates and keys used throughout your Rhapsody configuration.



The Certificate Manager allows the user to:

- Manage SSL Certificates and associated private keys, and certificates associated with keys for remote hosts.
- Manage Symmetric Keys.
- Manage SSH Public and Private Keys..
- Manage OpenPGP Pubic Keys and Private Keyrings to be used with the RPG Rhapsody filters.

Managing SSL Certificates

Generating new SSL Certificates

It is possible to generate new private keys and the associated self-signed certificate from the SSL Private Keys tab by selecting Generate New. This will open the Generate New Certificate dialog:

Generate New Certificate

Certificate Subject Name

Common Name

Organization Unit

Organization

Locality

State

Country or Region

Certificate Parameters

Key Algorithm Key Size (bits)

Signature Algorithm

Expiry Date

☐ Sign using existing private key

Certificate Usage

Certificate DNS Names

Enter the DNS names to include in the certificate (one per line)

New Certificate Location

Filename [Browse](#)

Password

Confirm

The **Certificate Subject Name** and **Certificate DNS Names** fields are used by external systems to help identify the owner of the certificate and determine what trust should be placed in the certificate. While the Common Name is required, other fields should be filled in as appropriate. As much information as possible should be provided as it may be reviewed by the user or external system to help assign trust to the public certificate.

The **Certificate Parameters** options determine the algorithm, complexity (key size) and validity duration for the new certificate. The details entered here will depend on the capability of the external systems. The default settings on the dialog should be considered as the minimum recommended secure settings. You can sign the certificate using an existing key to establish a trust relationship between an existing private key (provided signing of certificates was allowed when the private key was created) and the new certificate.

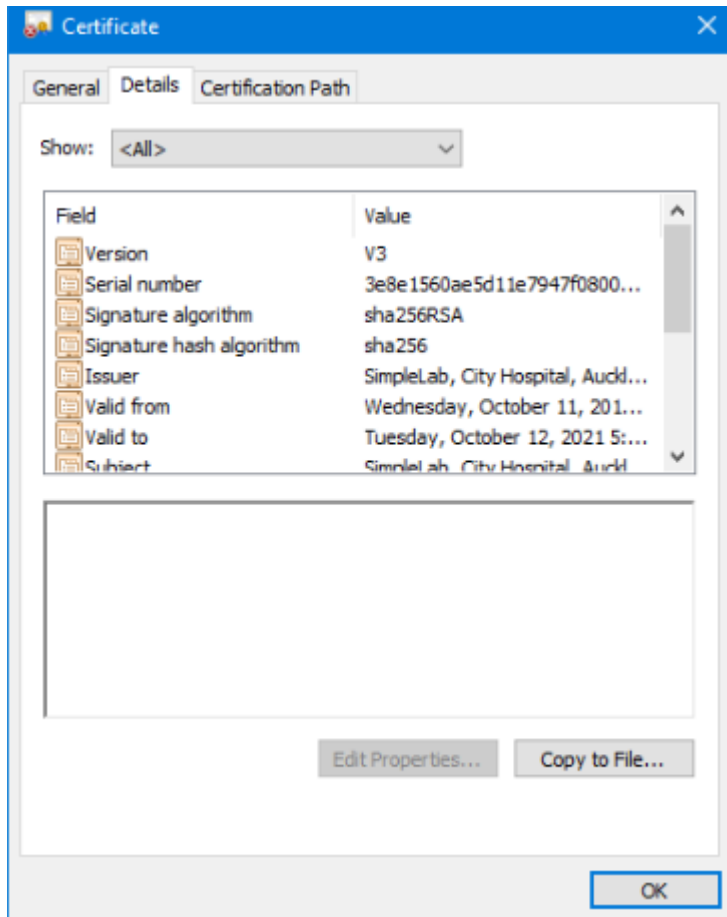
Rhapsody supports the SHA-1 signature algorithm (referred to as SHA1withRSA and SHA1withDSA in the Signature Algorithm drop-down). This algorithm contains known cryptographic weaknesses and should not be used unless absolutely required by an external system.

The **New Certificate Location** settings define a PKCS#12 file where the newly generated private key and certificate will be saved. As this file contains the private key, the password selected should be of an appropriate complexity.

Selecting **Generate** once all settings have completed will generate both the private key and associated certificate and save them to the specified file. Rhapsody will automatically prompt to import the newly generated certificate. Once imported, the certificate will appear under the **SSL Certificates** tab and the private key under the **SSL Private Keys** tab.

Distributing a Certificate

For an SSL key to be useful, the corresponding SSL certificate must be exported and sent to any system that sends SSL encrypted messages to Rhapsody. To export a certificate, select the **SSL Certificates** tab, then select **Export**. This will generate an X.509 formatted certificate that can be distributed to external systems. Details of the exported certificate can be viewed by opening the certificate from Windows.



The PFX file created when the private key was generated should not be distributed as it contains both the certificate and the private key. Distributing this file would allow any external system to decode messages being sent to Rhapsody.

Importing Certificates

The Certificate Manager allows the import of certificates for both the server and for remote hosts (public keys). Private keys generated outside Rhapsody may also be imported. This may be the case when the generated certificate must be certified against a Certificate Authority.

The process of obtaining a certificate from a Certificate Authority for a server can be outlined as follows (a Certificate Authority will provide documentation on the exact details of the steps they require):

- Generating the private and public keys for the server.
- Generating the certification request for the server and sending to the authority (note that this includes the organization details and the public key only).
- Receipt of the certificate.
- Import to the server.

Certificates from remote systems should be supplied in the **DER** format, typically in a file of type .cer. Note that certificates may be supplied in other formats, and it may be necessary to convert the certificate to the DER format. The [OpenSSL\(opens in a new tab\)](#) tools provide the ability to translate between formats.

Import formats recognized by the Certificate Manager are:

- .cer: binary format certificate
- .pfx: PKCS#12 binary multipart certificate generated on Windows (contains certificate and a private key)
- .p12: PKCS#12 binary multipart certificate generated on Linux / Unix host (contains certificate and a private key)

Certificate Chains

Certificate Authorities are recognized as root authorities or intermediate authorities. The root authority is recognized as one of the validating organizations, while an intermediate derives their authority from the root authority (possibly via one or more additional authorities).

When a certificate is issued by an intermediate authority, the user must be able to trace the chain of validation back to the root authority; consequently, the intermediate certificates defining that chain of authority must also be provided.

The certificate chain should be defined in the sequence of the chain, but it may not be. In these cases, the Certificate Manager may experience difficulty in validating the certificate.

Managing SSH Certificates

SSH keys cannot be generated from within Rhapsody. They must be generated by an external tool and imported. Rhapsody supports keys in either the OpenSSH, ssh.com, or PuTTY formats. There are tools for each of the three formats that can be used to generate SSH public and private keys.

The following resources provide additional background around SSH key generation:

- <https://help.ubuntu.com/community/SSH/OpenSSH/Keys>(opens in a new tab)
- http://winscp.net/eng/docs/ui_puttygen(opens in a new tab)

Importing Keys

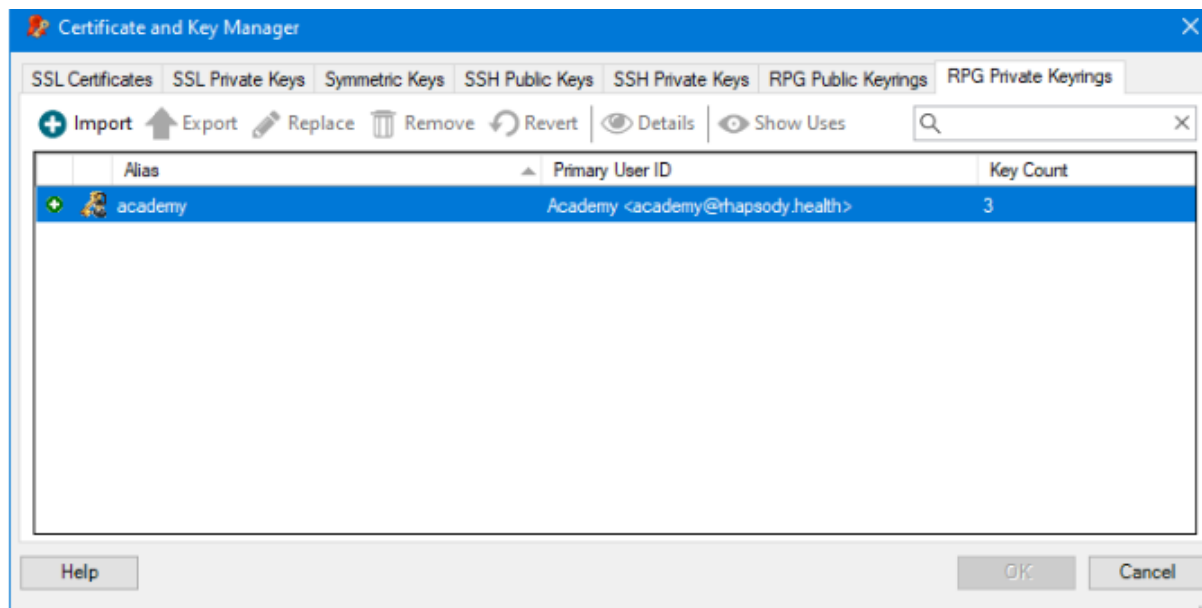
SSH keys are imported into Rhapsody from either the **SSH Public Keys** or the **SSH Private Keys** tab. Select the **Import** button to start the process. You will be prompted for the name of the file containing the key to import along with an alias to be used in Rhapsody and, for private keys, the password needed to access the key.

Managing RPG Certificates

RPG (Rhapsody Privacy Guard) is Rhapsody's implementation of a subset of the [OpenPGP](#)(opens in a new tab) standard. The RPG filters in Rhapsody can be used for decrypting and/or encrypting messages. As part of the filters' configuration, the RPG keyrings are used as part of the configuration.

Keyrings cannot be generated from within Rhapsody. They must be generated by an external tool and imported before being utilized in the RPG filters. The following resources provide additional background around OpenPGP keyring management:

- <https://gnupg.org/software/frontends.html>(opens in a new tab)
- https://userbase.kde.org/Concepts/OpenPGP_Getting_Started



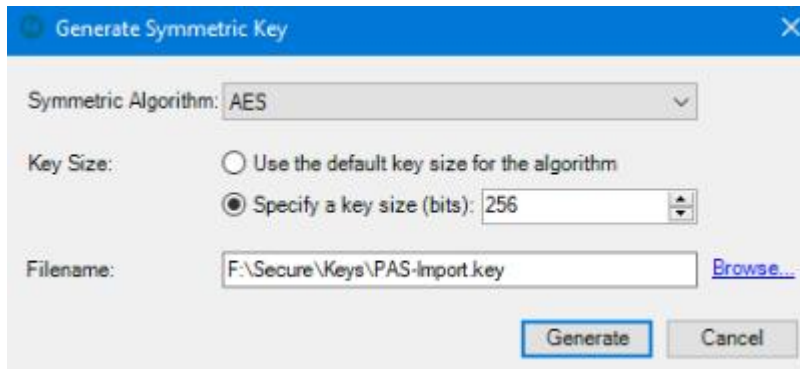
The **RPG Keyring Details** dialog displays the **Alias**, **User ID** and **Type** for the keyring and the following information for each of its associated keys:

Field	Description
Key ID	The OpenPGP key id.
Signing Key	Whether the key can be used for messages signing.
Encryption Key	Whether the key can be used for message encryption.
Algorithm	The encryption algorithm used to generate the key.
Bit Strength	The bit strength of the encryption algorithm used
Expires	The date when the key expires. If the key has no expiry date, a value of Never appears in this field. If the key is expired, it is displayed in red.
Created	The date when the key was created.
Users	Name and email address of the user associated with this key.

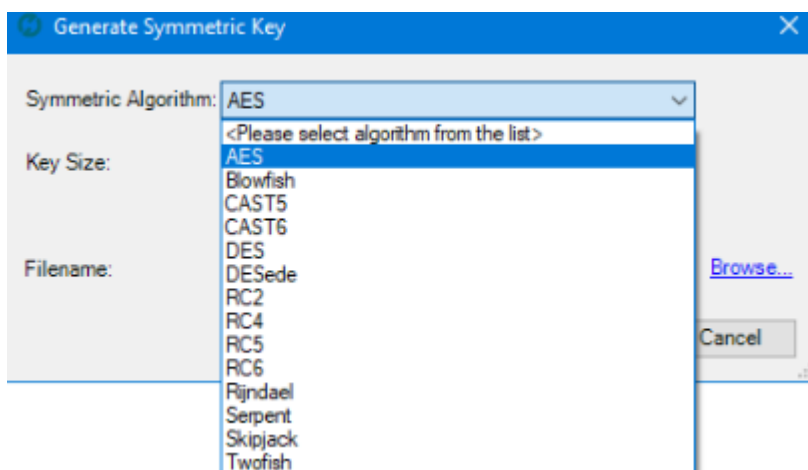
Managing Symmetric Keys

Generating New Symmetric Keys

It is possible to generate new symmetric keys from the **Symmetric Keys** tab by selecting **Generate New**. This will open the **Generate Symmetric Key** dialog:



An algorithm needs to be selected for the key based on the capabilities of the external system Rhapsody is communicating with. Generally, a bigger **key size** will equate to a higher degree of security, however, this varies from algorithm to algorithm and may be constrained by the capabilities of the external system.



Selecting **Generate** will generate both the key and save it to the specified file. Rhapsody will automatically prompt to import the newly generated key. Once imported, the key will appear under **Symmetric Keys** tab.

Importing and Exporting Keys

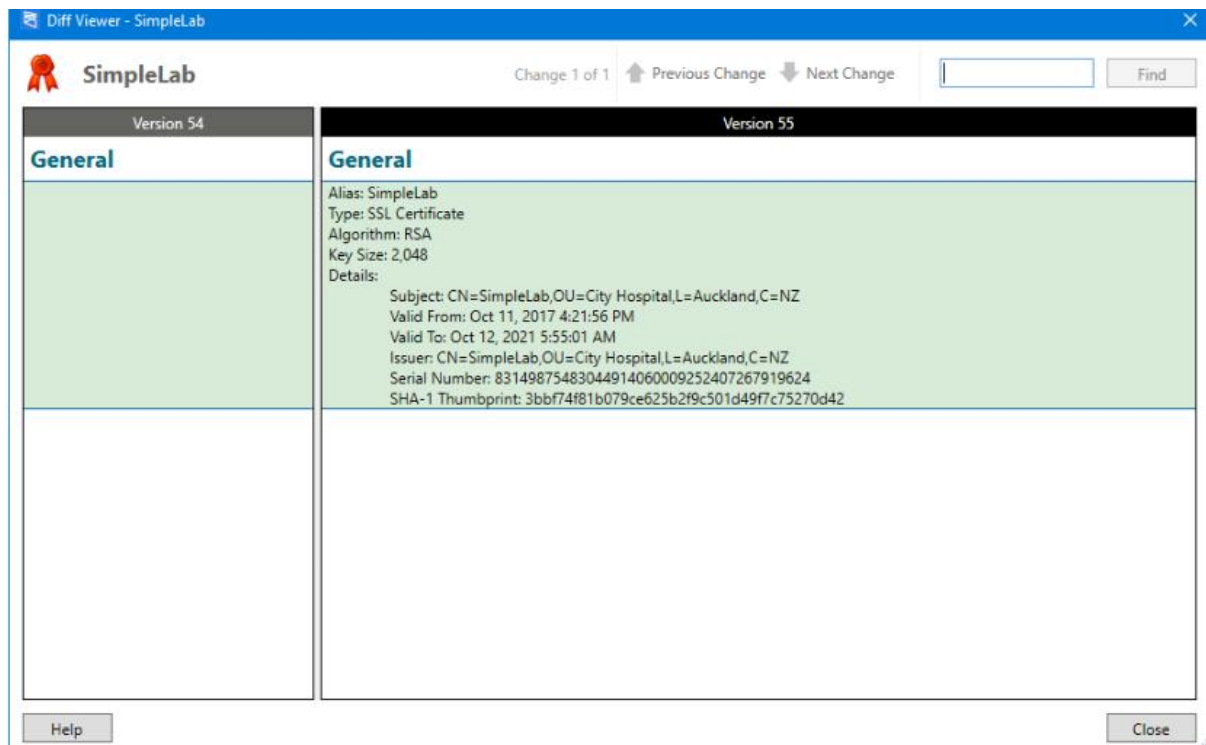
Symmetric keys generated by other systems can be imported by selecting the **Symmetric Keys** tab by selecting **Import**. Keys stored within Rhapsody can be exported to other systems by selecting **Export** from the same tab.

There is no access protection for symmetric keys. Anyone who is able to access a key will be able both read and generate valid messages. For this reason, symmetric keys must be carefully protected.

Check-in & Migration

Once changes made via the **Certificate and Key Manager** are complete, they need to be checked in. This task is performed by exiting the manager and selecting the **Check In** option from the toolbar via the Rhapsody menu. As with other components, the check in screen will detail all items that have been added, changed or deleted.

As with other components, details of a change can be viewed from the **Diff Viewer**.



Migration

When you export your configuration in an RLC, certificates and keys are not included. It is expected that the configuration will use a different key (with the same name) in each environment where it is deployed. If the same key is required in multiple environments, it will need to be exported from the **Certificate and Key manager** and manually migrated to the destination environment.

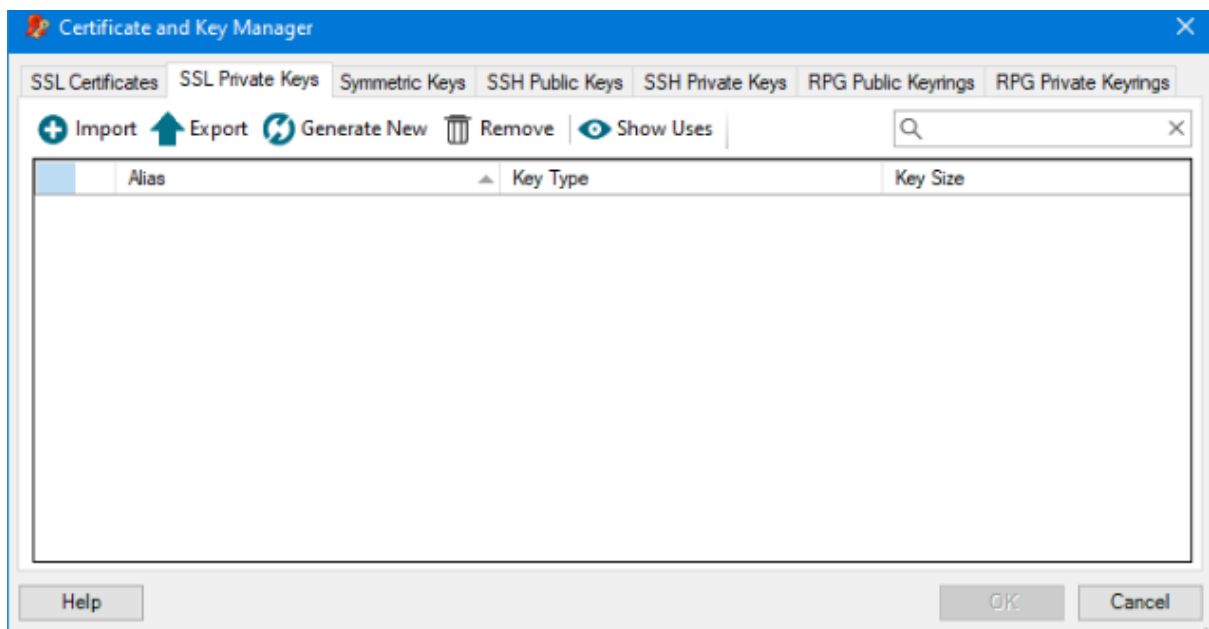
Exercise - Certificate Management

Overview

This exercise reviews the process of generating a private SSL key for a Rhapsody service, and importing a public certificate from a remote service. These credentials will be used in later exercises.

Generating a Self-Signed Key

Login to the Rhapsody IDE and select the **Certificates and Key Manager** from the View menu. This will display the Certificates and Keys Manager. Select the SSL Private Keys tab to display the list of private SSL keys installed in this Rhapsody instance. On a new Rhapsody server, this list will be empty.



Select **Generate New** to start the process of creating a new self-signed private key. This will display the **Generate new Certificate** window. Under the **Certificate Subject Name** section, enter your details. The default settings under the **Certificate Parameters** section are correct for this test certificate.

Under the **New Certificate Location** section, select **Browse** to select a file to save the new certificate to and enter a password for the new certificate. In a production environment, it is very important that this password is securely selected and managed to ensure that only authorized users are able to access the private key. It is also important that the password to the certificate is not lost.

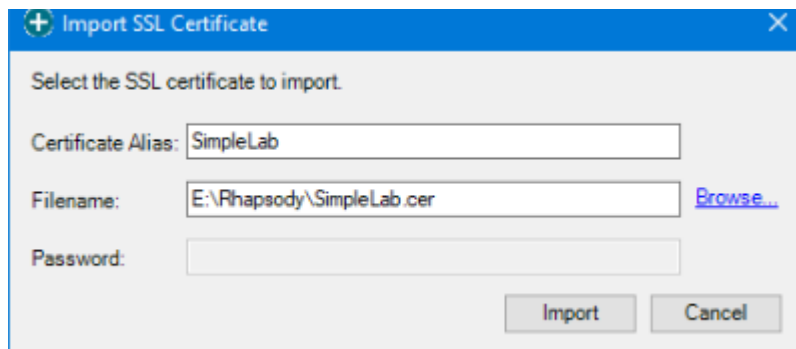
Once all details have been entered and confirmed, select **Generate**. Rhapsody will generate the key, then on success ask you if the new key should be added to the certificate store. Select **Yes** to add the newly generated key, this will display the **Import SSL Private Key** dialog. This is the same dialog that appears when importing an SSL private key from an external source (although with the Filename option enabled).

When prompted, enter an alias for the new key, ensure you choose something that reflects the intended use for the key (in this case it will be for training). Notice how you are required to enter the password for the key during import. Rhapsody must know the password for the private key to be able to open the key when a message needs to be decrypted.

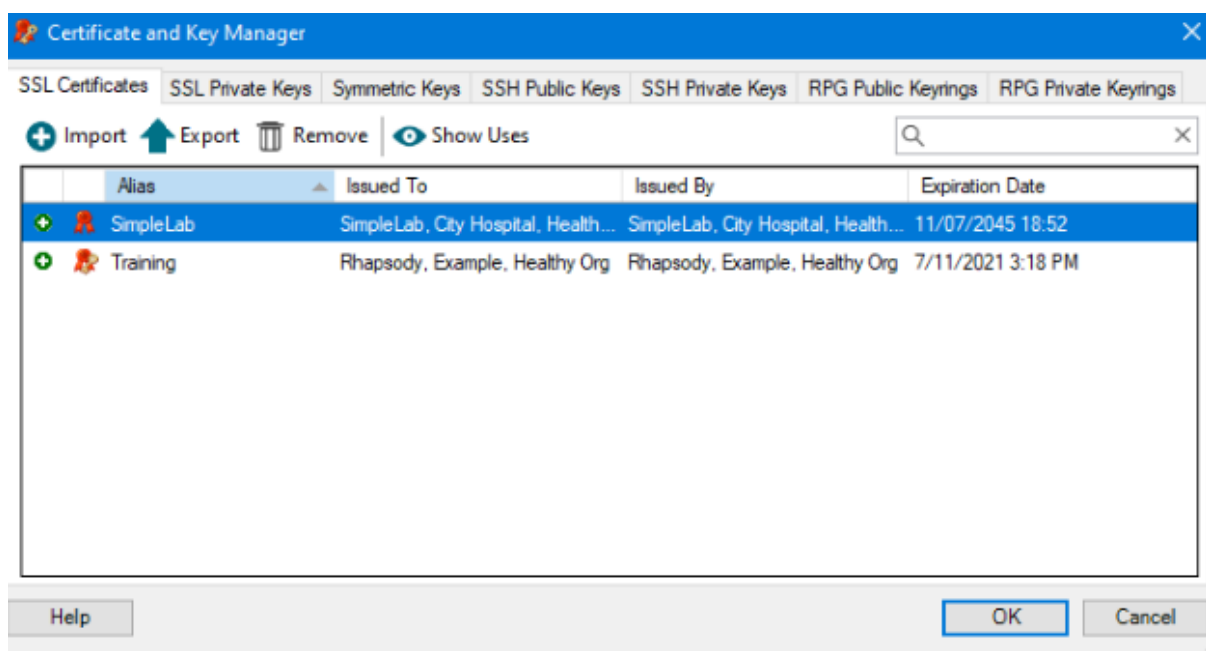
The new private key will appear under the **SSL Private Keys** tab, while the associated certificate will appear under the **SSL Certificates** tab.

Importing a Certificate

Select the **SSL Certificates** tab of the **Certificates and Key Manager** screen.
Select **Import**, then select **Browse** to select the certificate downloaded above, then select **OK** to complete the import.



The imported certificate will be displayed alongside the self-signed certificate generated previously in this exercise. Notice how the self-signed certificate has the same details in the Issued To and Issued By fields, while the imported Wikipedia certificate has been issued by a third party who we should trust before using the certificate.



Finally, close the Certificate and Key Manager and check-in the changes to commit them to the Rhapsody engine.

Lookup Table Management

Overview

Often there is a need to substitute one code in a message with one or more other codes. In Rhapsody, this can be done with a Lookup table. For example, a message that contained an address with a ISO3166 2 digit country code could be mapped to the country name using the lookup service.

A lookup table is effectively an embedded table which is incorporated into the Rhapsody configuration and managed by tools provided in the IDE and the Management Console. This eliminates the need for more complex table management in a database, and supports management of tables from within the Management Console by users authorized to access the Lookup Tables.

The Lookup Service manages all Lookup tables. Each Lookup table will typically contain two or more columns.

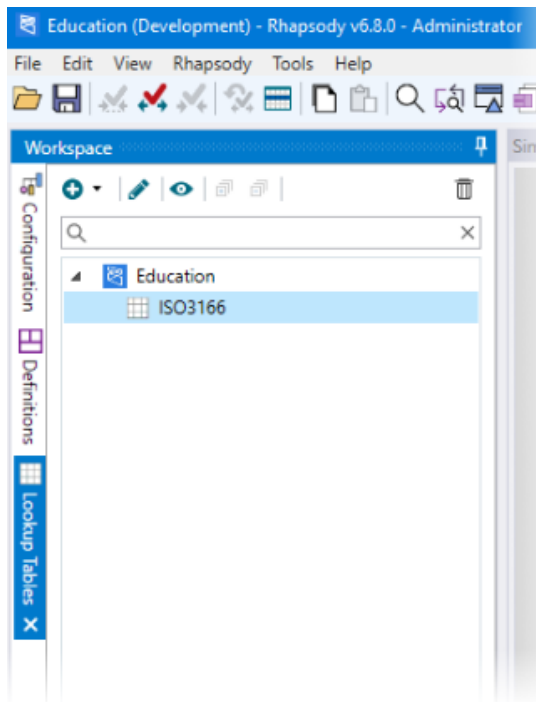
Lookup tables can be accessed from the following filters:

- JavaScript
- Message Modifier
- Mapper

Creating a New Lookup Table

Lookup Tables are created and managed from the **Lookup Tables** tab in the **Workspace** of the Rhapsody IDE. This tab is accessible from the **View** menu or the **Managers** drop-down.

Lookup tables are managed either from the ribbon at the top of the Workspace, or by right-clicking within the Lookup Tables tab.



Creating a New Lookup Table

A new Lookup Table is created by selecting the **Add** button at the top of the Workspace screen.

The screenshot shows the "Lookup Table Structure" dialog box. It has a title bar with a grid icon and a close button. The dialog contains a "Table Name:" text field, a "Description:" text area, and a checkbox labeled "Record lookup failures" which is checked. Below these is a "Column Definition" section with a table. The table has four columns: an unnamed column, "Name", "Indexed", and "Default Value". The first row of the table contains an asterisk (*) in the first column, a small icon in the "Name" column, an unchecked checkbox in the "Indexed" column, and an empty "Default Value" cell. Below the table is a large grey rectangular area. At the bottom of the dialog are three buttons: "Help", "OK", and "Cancel".

	Name	Indexed	Default Value
*		☐	

Enter a **Table Name**. The name will be used to refer to the Lookup Table throughout the rest of Rhapsody, so should be something that succinctly describes the content of the table. A **Description** should be included as part of the solution documentation.

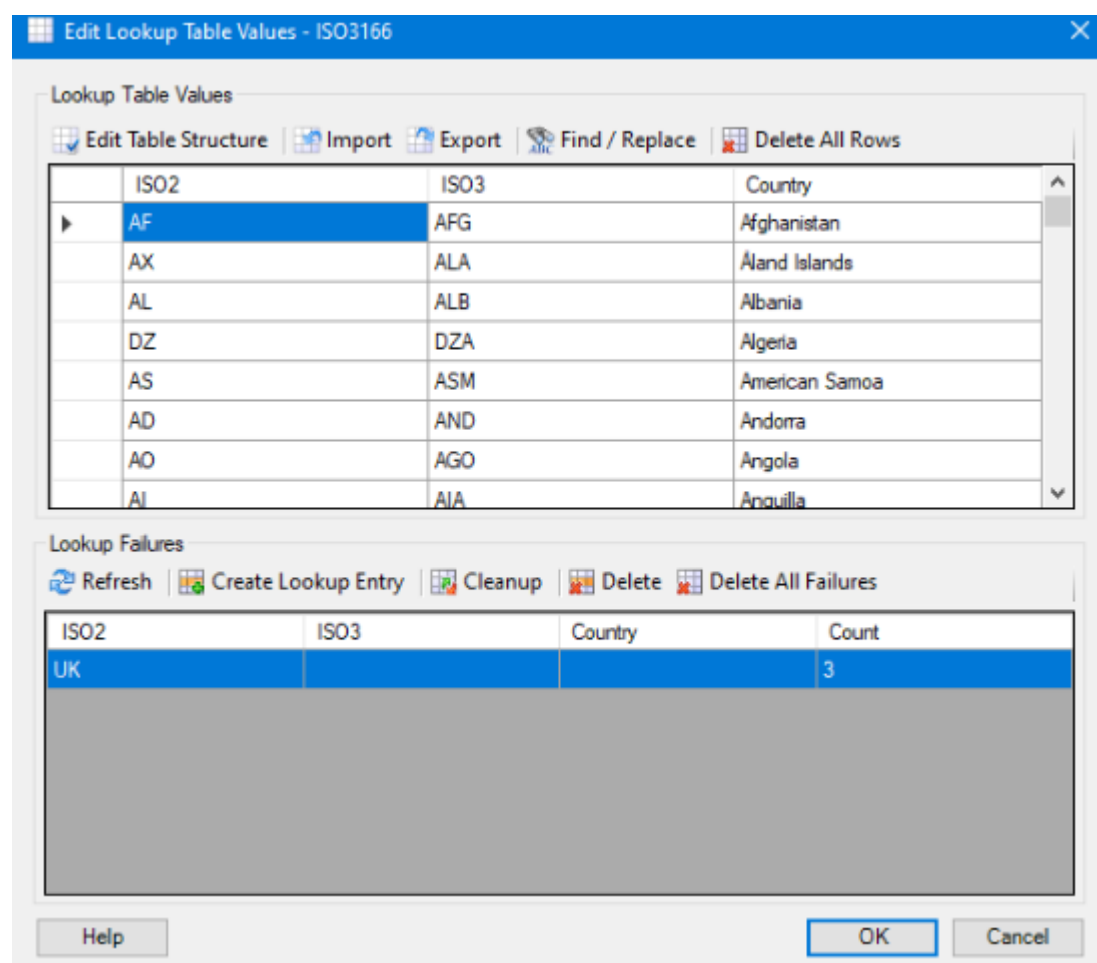
The **Record lookup failures** checkbox instructs Rhapsody to record all failed lookups against the table. This is useful when the lookup table is used to modify messages generated by an external system that may occasionally send unexpected codes.

Next, a number of columns needs to be defined. A column that has been added in error can be deleted by right-clicking on the row. The **Indexed** option will improve the search performance of a column that is frequently searched in a large table. The **Default Value** provides the default value to return for a record where the column does not have a value.

Editing Data in a Lookup Table

Entries can be added to a Lookup Table in one of three ways:

- By performing a bulk import from a CSV file
- By manually editing the Lookup Table
- By adding previously failed lookup attempts



Bulk Import

A Lookup Table can be populated with a CSV file. The first row of the CSV file must contain column headers that match the already defined column headers in the Lookup Table. All subsequent rows will be considered content to be added to the Lookup Table.

This is the quickest way of adding data to a lookup table when it has already been defined externally. The imported data can be configured to either overwrite the existing contents of the lookup table, append to the existing lookup table data, or be merged in.

Manually Editing a Table

An entry can be manually added to the Lookup Table by scrolling to the end of the table and entering values directly into the table.

Lookup Failures

The Lookup failures table lists all the code lookup attempts against the current lookup table that have failed. It also provides a number of options for managing the failures.

If the **Record lookup failures** option was selected when the Lookup Table was created, you will be able to see all lookup failures that have occurred in Rhapsody against this table.

The **Count** column indicates how many times the particular lookup failure occurs.

The **Create Lookup Entry** button adds the failed lookup into the Lookup Table. You will need to manually complete the other columns for the lookup.

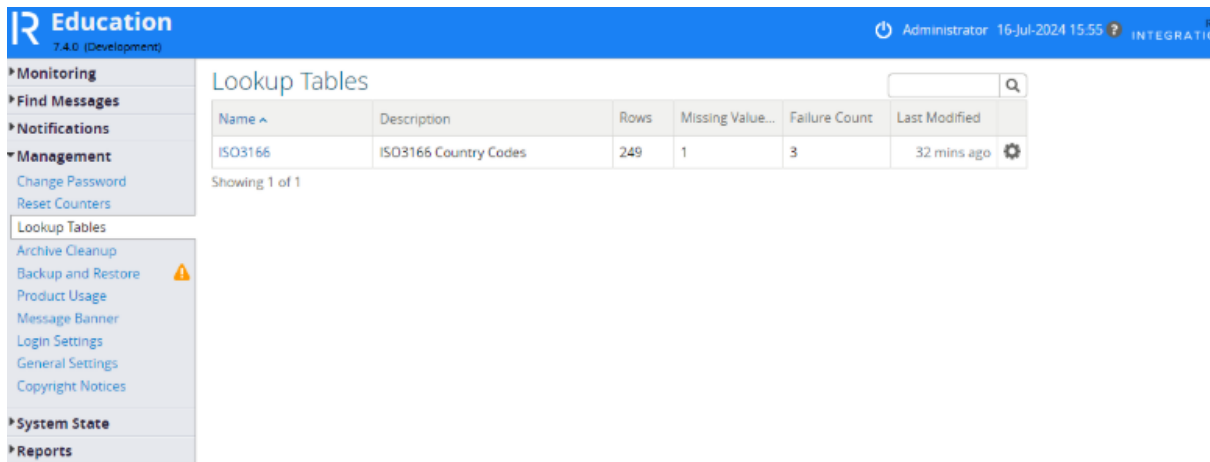
The Cleanup button removes rows from the lookup failures table which have been added into the Lookup Table.

Check in

Once changes to the Lookup Table are complete, you will need to check in your changes before they take effect.

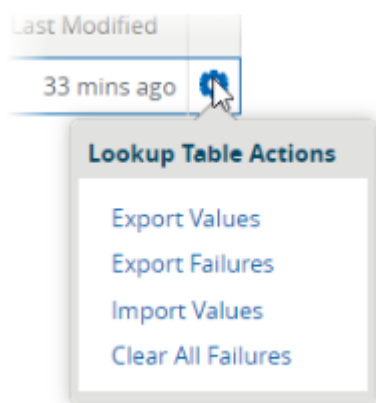
Managing a Lookup Table

Lookup Tables are managed from the Rhapsody Web Management Console. To view a list of all Lookup Tables in Rhapsody, select the **Lookup Tables** item from the **Management** menu.



Hovering the mouse over the cog at the right end of each lookup row will display a pop-up of common **Lookup Table Actions**.

Selecting the name of a Lookup Table in the list will display the details for a Lookup table, including detailed information on failure counts. When viewing the details for a Lookup Table, the same options available via the **Edit Lookup Tables Values** screen (discussed in the previous page) are available.



Using a Lookup Table

You can use a Lookup Table from the JavaScript, Message Modifier, Mapper, and FreeMarker filters. Lookup tables can also be used as part of conditional connector rules.

Conditional Connectors

Lookup tables can be referenced with the **Found in lookup table** and **Not found in lookup table** operators.

JavaScript

Lookup tables can be accessed from JavaScript using the lookup function:

```
lookup(tableName, column, value [,column2, value2, ...]);
```

Where:

- tableName is the name of the Lookup table to use
- column is the column in the Lookup table to search against
- value is the value to search for in the column specified above

The function returns an array containing all the columns for each matching row, or Null if the Lookup failed.

```
const countryCode = next.getField("PID.PatientAddress[0].Country");
```

```
//Simple lookup
```

```
const result = lookup("ISO3166", "ISO3", countryCode);
```

```
if (result != null) {  
    next.setProperty("CountryName", result.Country);  
    next.setProperty("CountryCode", result.ISO3);  
} else {  
    //Lookup failure  
    next.setProperty("CountryName", "");  
    next.setProperty("CountryCode", "");  
}
```

Message Modifier

To use a Lookup Table from the Message Modifier, when adding an action, select the **Using a Lookup Table** option.

Change Field ▼ PID.11.6

Using a Lookup Table ▼

Table Name IS03166

Search Column IS03

Using Value From Field ▼ PID.11.6

Replace With Value in Column Country

If No Value Found Then Use a Default Value ▼

Default Value

OK | Cancel

Mapper

When the Mapper is being run from within Rhapsody, the Rhapsody lookup tables are available via the `RhapsodyTableLookup` function.

`RhapsodyTableLookup(output, tableName, resultColumn, defaultValue, column, value [,column2, value2, ...]);`

Where:

- `output` is the field in the output message or string variable to be modified
- `tableName` is the name of the Lookup table to use
- `resultColumn` is the column in the Lookup table that contains the result
- `defaultValue` is the default value to output if the lookup fails
- `column` is the column in the Lookup table to search against
- `value` is the value to search for in the column specified above

More details on the use of the Mapper to perform a lookup can be found in the Rhapsody Help documentation (search for "Using Map Designer for Lookups").

FreeMarker Mapper

Lookup tables can be inserted into the output of a FreeMarker filter using the `lookupTable` function.

```
${lookupTables.get("table")}
```

```
${lookupTable.lookup("column", "value", "resultColumnName")}
```

Where:

- `table` is the name of the Lookup table to use
- `column` is the column in the Lookup table to search against
- `value` is the value to search for in the column specified above
- `resultColumnName` is the column in the Lookup table that contains the result